

Capstone Project III: Web Scraping Data Pipeline & Price Intelligence Analytics

1. Project Objective

Following the introduction to web scraping techniques and ETL pipeline development, the objective of this capstone is to design and implement a robust, scalable web scraping data pipeline that extracts product data from real-world websites and transforms it into a structured, analytics-ready database.

You are required to:

- Implement a web scraping pipeline
- Apply data cleaning and transformation techniques
- Design a structured PostgreSQL data model
- Generate insight-driven SQL outputs

2. Business Context

ShopEasy Retail Intelligence is a company focused on monitoring pricing and market trends across multiple platforms.

Currently, data is manually collected, leading to:

- Inconsistent pricing information
- Limited visibility into trends
- Poor decision-making

You are required to build a system that automates data collection and enables analytics.

3. Specialized Tasks

Task 1: Data Extraction

Scrape product data from real websites. Eg(books.toscrape.com - Built for learning, no restrictions, or quotes.toscrape.com - Educational, good for pagination)

Extract: product name, price, rating, category.

Ensure your solution handles real-world website behavior.

Task 2: Data Cleaning & Transformation

- Clean price values
- Standardize ratings
- Handle missing values

- Generate unique IDs
- Remove duplicates

Task 3: Database Design & Loading

Create a PostgreSQL schema and structured tables.

Define primary keys and constraints.

Load transformed data into the database.

Task 4: Business Analytics

Write SQL queries to answer:

- Which products are overpriced or underpriced?
- Which categories show price variation?
- How do prices change over time?
- What insights can be derived?

Task 5: Pipeline Automation (Optional)

Implement scheduling, logging, and retry mechanisms.

4. Expected Data Model

Design a structured schema with relevant tables for storing scraped data.

5. robots.txt Requirement (IMPORTANT)

Before scraping any website, students **MUST** check the robots.txt file. A robots.txt file defines rules for bots and scrapers.

Example location:

[https://\[website.com\]/robots.txt](https://[website.com]/robots.txt)

Look for:

- Disallow rules (pages you cannot scrape)
- Crawl-delay (wait time between requests)
- User-agent rules (applies to all bots)

Implementation:

- Add `time.sleep()` delays in code
- Respect all Disallow rules
- Add User-Agent header
- Handle 429 (Too Many Requests) responses

This ensures ethical and responsible scraping practices.